Computer Networks — Exam Question Bank (with answers)

KU Leuven · G0Q43A Computer Networks (Danny Hughes) · compiled for the exam of **Mon 8 June 2026**. Sources: the curated 106-question bank (`Examen_CN_examenvragen.pdf`, all unique questions 2015–2025), the 2023 open-book model answers, the Ward Coomans/Kevin Vandeputte all-years collection, Jasper Visterin's embedded exam questions, and the A-summaries (Tiemen Moeyersons, Jasper Visterin). Answers are written for understanding, not memorisation, and reworded — verify numbers yourself in the open-book part.

How to use this file

- Questions are grouped **by layer of the Tanenbaum/TCP-IP stack**: Application → Transport → Network → Data Link & Physical.
- Each question has a short `###` title, a `####` line naming **which layer(s) it applies to** (cross-layer questions are flagged), and a **Tags:** line of protocols/principles you can `Ctrl+F`.
- `Book:` says whether it has historically appeared closed-book (CB), open-book (OB), or both. `Seen:` gives provenance.
- **Exam trap** and **Hughes emphasis** are called out where they matter.
- Open/closed duplicates from the source bank are merged. Items the course **dropped this year are not answered**: *Chord* (Gnutella-over-Chord) and *Wake-on-LAN* — Jasper notes both are "not studied this year". *Leaky/token bucket* appears only in a 2015 paper and is likely de-emphasised (one-line note in Transport).

Tag index

Layering & general: `OSI` · `TCP/IP-stack` · `Tanenbaum-stack` · `layering` · `connection-oriented` · `connectionless`

Application: `DNS` · `recursive-query` · `iterative-query` · `caching` · `anycast` · `DHCP` · `BOOTP` · `Gnutella-0.4` · `Gnutella-0.6` · `super-peer` · `PING-PONG` · `QUERY-QUERYHIT` · `flooding` · `TTL` · `PUSH` · `NAT-traversal` · `Napster` · `BitTorrent` · `P2P` · `FTP` · `active-mode` · `passive-mode` · `HTTP` · `pipelining` · `persistent-connection` · `TOR` · `onion-routing` · `RTP` · `SMTP` · `Telnet` · `broadcast` · `multicast`

Transport: `TCP` · `UDP` · `QUIC` · `TCP-header` · `three-way-handshake` · `flow-control` · `sliding-window` · `window-probe` · `congestion-control` · `AIMD` · `slow-start` · `ssthresh` · `TCP-Tahoe` · `TCP-Reno` · `fast-recovery` · `SACK` · `ACK-clock` · `go-back-N` · `selective-repeat` · `NACK` · `stop-and-wait` · `tinygram` · `Nagle` · `silly-window` · `Clarke` · `delayed-ACK` · `ECN` · `RED` · `choke-packets` · `TSAP` · `port` · `leaky-bucket`

Network: `IPv4` · `IPv6` · `IP-addressing` · `subnetting` · `CIDR` · `subnet-mask` · `broadcast-address` · `default-gateway` · `routing-table` · `ARP` · `NAT` · `MTU` · `path-MTU-discovery` · `fragmentation` · `segmentation` · `ICMP` · `DF-bit` · `TTL-field` · `OSPF` · `BGP` · `autonomous-

system · AODV · distance-vector · link-state · count-to-infinity · split-horizon · RPL · Zigbee · BLE · IoT · SLAAC · autoconfiguration · EUI-64 · DHCPv6 · checksum · NSAP

Data Link & Physical: Ethernet · switch · hub · CSMA/CD · CSMA/CA · collision-detection · padding · cable-length · full-duplex · half-duplex · ALOHA · slotted-ALOHA · 802.11 · WiFi · MAC-protocol · hidden-terminal · exposed-terminal · RTS/CTS · MACA · channel-hopping · BMAC · LPL · preamble · sampling-interval · duty-cycle · TSMP · TDMA · time-synchronization · LoRa · null-MAC · byte-stuffing · bit-stuffing · framing · beacon-frame · TIM · PS-Poll · APSD · power-saving · RTT · throughput · bandwidth-delay-product · satellite · frame-loss · promiscuous-mode

---

Application Layer & Layering

A-L1 — Draw the OSI, TCP/IP and Tanenbaum stacks; one protocol per Tanenbaum layer

Layers: Layering (Application + all)

Tags: OSI TCP/IP-stack Tanenbaum-stack layering **Book:** CB · **Seen:** bank Q1; 2018, 2025-06-10, 2025-06-24

OSI (7)	TCP/IP (4)	Tanenbaum (5)	Example protocol
Application	Application	**Application**	HTTP, DNS, DHCP, FTP, SMTP, Gnutella, RTP
Presentation	”	(folded into App)	—
Session	”	(folded into App)	—
Transport	Transport	**Transport**	TCP, UDP
Network	Internet	**Network**	IPv4/IPv6, OSPF, BGP, AODV
Data Link	Link	**Data Link**	Ethernet, 802.11, ALOHA, BMAC, TSMP
Physical	(in Link)	**Physical**	copper, fibre, radio

Why three models: OSI came from telecom carriers who wanted **connection-oriented**, reliable service, so it has fine-grained Presentation/Session layers. TCP/IP came from ARPAnet, designed to **survive failures**, so it is leaner. Tanenbaum is the teaching compromise: it keeps the lean 5 layers but, unlike pure TCP/IP, splits Link into Data Link + Physical and folds Presentation/Session into Application. **Exam trap:** give a *protocol* per layer, not a device. Hubs/repeaters live at Physical, switches/bridges at Data Link, routers at Network.

A-L2 — Which OSI layer does each protocol belong to?

Layers: Layering

Tags: `OSI` `layering` `TCP` `UDP` `SMTP` `FTP` `DHCP` `Gnutella-0.4` `BGP` `RTP` `AODV` **Book:** CB · **Seen:** bank Q2; Toledo sample, 2020, 2021

TCP → Transport (4). UDP → Transport (4). SMTP → Application (7). FTP → Application (7). DHCP → Application (7). Gnutella → Application (7). BGP → Network (3). AODV → Network (3). RTP → "officially" Application (7) but defensibly Transport (4) — see A-L3. **Note:** the question is usually phrased in the 7-layer OSI model even though the course teaches the 5-layer Tanenbaum model; the layer numbers above are OSI.

A-L3 — RTP and DHCP: which layer, and the argument for placing them elsewhere

Layers: Application & Transport (RTP); Application & Network (DHCP)

Tags: `layering` `RTP` `DHCP` `UDP` **Book:** CB · **Seen:** bank Q3; Toledo, 2020, 2021

RTP — officially **Application**, arguably **Transport**. - *Application:* it is not built into the OS kernel like TCP/UDP — it ships inside the media application (user space), and it runs *on top of* UDP. - *Transport:* it provides a **generic transport service** (sequence numbers, timestamps, multiplexing of media streams) that is not tied to one specific application.

DHCP — officially **Application**, arguably **Network**. - *Application:* it uses the transport layer (UDP) and a client–server request/response model, exactly like other app-layer services. - *Network:* its whole job is handing out **IP addresses** and network config — the core concern of the network layer, and it interacts with machines rather than users.

A-L4 — Connection-oriented vs connectionless + one example per layer

Layers: Layering (Application, Transport, Network, Data Link)

Tags: `connection-oriented` `connectionless` `layering` `three-way-handshake` **Book:** CB · **Seen:** bank Q4; 2022, 2022-06-28

	Connection-oriented	Connectionless
Setup	handshake first (e.g. TCP 3-way)	none — just send
Reliability	ordered, error-controlled, flow/congestion control	best-effort, maybe a checksum
Broadcast	no	natural fit
Cost	higher overhead	fast, simple

Example protocols (connection-oriented / connectionless): - **Application:** SSH or SMTP / DHCP or DNS or HTTP - **Transport:** TCP / UDP - **Network:** *OSPF (session-like)* / AODV (per-the-course framing; note real IP itself is connectionless) - **Data Link:** point-to-point / serial line / ALOHA, Ethernet, 802.11

Exam trap (Hughes loves this): *connectionless at the application layer ≠ connectionless transport.* HTTP/1.0 is "stateless/connectionless" at the app layer yet runs over **connection-oriented TCP** — each object opens a fresh TCP connection. So an app-layer protocol can be connectionless while its transport is connection-oriented. Likewise you almost never build connection-oriented *on top of* connectionless except by adding it yourself (RTP/QUIC over UDP).

A-DNS1 — Why split DNS into a recursive and an iterative phase?

Layers: Application

Tags: `DNS` `recursive-query` `iterative-query` `caching` **Book:** CB · **Seen:** bank Q5/Q9; 2014, 2018, 2022, 2025-06-24

The client sends **one recursive query** to its **local DNS server**: "give me the full answer, no half-answers." The local server then runs the **iterative phase** down the name hierarchy — it asks the root, which refers it to the `.com` server, which refers it to the `google.com` server, which returns the address. Iterative servers may answer **partially** (a referral to a lower server) rather than fully.

Why the combination beats either end-to-end: - **Load distribution:** the few queries hitting the root/`.com` servers each carry tiny load (just a referral); the heavier recursive work sits on the many local servers. No server must store every IP in the world. - **Caching:** the local server caches answers (with a TTL), so popular sites resolve instantly for a whole region — impossible in a purely iterative design. - **Simplicity for the client:** the end host issues one query and gets a final answer instead of chasing referrals itself. - **Hierarchy / fault tolerance:** each server only owns its slice of the namespace; if one path fails the resolver can try another.

A-DNS2 — What would a *purely* iterative or *purely* recursive DNS look like? Pros/cons

Layers: Application

Tags: `DNS` `recursive-query` `iterative-query` `caching` **Book:** CB · **Seen:** bank Q7/Q8; 2014, 2022-06-28

Purely iterative (no local server; the client chases every referral): the client must replicate a resolver's logic and do many round trips, with extra latency if it is far from the servers, and it **cannot benefit from a shared local cache**. Advantage: no local-server dependency.

Purely recursive (the local server must return the final answer for everything, end to end): the load and memory burden on that one server explode — to never return a referral it would effectively need to know/resolve every name. Advantage: the client is trivial; one server "does the trick." This does not scale.

→ The mixed design exists precisely to get the client-simplicity of recursion *and* the scalability/caching of iteration.

A-DNS3 — Why does DNS use UDP instead of TCP?

Layers: Application & Transport

Tags: `DNS` `UDP` `TCP` `anycast` `Telnet` **Book:** CB · **Seen:** bank Q6; 2018, 2025-06-10, 2025-06-24

DNS exchanges are **tiny, short-lived, one-shot** request/response messages. A TCP 3-way handshake (plus teardown) per lookup — and per iterative hop — would multiply the overhead and latency for no benefit, and would load the already-busy name servers. UDP is "good enough": send a small datagram, get a small reply. Bonus: UDP enables **anycast**, letting many physical servers share one address for redundancy and attack resistance. If the reply is too large or truncated, DNS falls back to TCP. **Telnet sub-question (2015):** you *can* poke a DNS server with Telnet, but DNS is binary over UDP and Telnet is line-based over TCP, so it is awkward; the right tool is `dig`/`nslookup`.

A-DHCP — Describe DHCP operation; could it run over TCP?

Layers: Application (interacts with Network)

Tags: `DHCP` `UDP` `broadcast` `BOOTP` **Book:** CB · **Seen:** bank Q10/Q11; 2014, 2017, 2018-27, 2025-06-…

A host with no IP yet cannot use a connection-oriented protocol, so DHCP uses **UDP broadcast**: 1. **DHCP DISCOVER** — client broadcasts to `255.255.255.255` ("any DHCP server out there?"). 2. **DHCP OFFER** — one or more servers offer an IP (+ optionally subnet mask, default gateway, DNS server). 3. **DHCP REQUEST** — client picks one offer and requests it. 4. **DHCP ACK** — server confirms; the address is **leased** for a fixed time. After that the client can talk normally (incl. TCP).

Extras: lease renewal via **REQUEST** at ~50% lease elapsed (server replies **ACK**/**NACK**); **RELEASE** when done; **DECLINE** if the client's ARP probe finds the offered address already in use (duplicate-address detection). Predecessor was **BOOTP**.

Could DHCP use TCP? No — the client has no IP address and the message must reach *unknown* servers, so it must **broadcast**, which TCP (point-to-point, connection-oriented) cannot do. Leases also suit a lightweight, stateless exchange. **Hughes emphasis:** the lease-time trade-off — short leases reclaim addresses efficiently, long leases reduce server load.

A-GN1 — Gnutella 0.4: connection setup, search, termination, NAT, TTL

Layers: Application

Tags: `Gnutella-0.4` `PING-PONG` `QUERY-QUERYHIT` `flooding` `TTL` `PUSH` `NAT-traversal` `P2P` **Book:** CB · **Seen:** bank Q12; 2018, 2020, 2022, 2025-06-10

Unlike **Napster** (one central, legally-targetable index), Gnutella 0.4 is **fully decentralised** — every peer is equal, does discovery *and* maintenance, and only knows its immediate neighbours (a degree of anonymity).

- **Connection setup — PING/PONG:** a joining node sends a **PING** to a known node; it is **flooded** outward. Any node willing to accept a connection replies with a **PONG** (carrying its address) routed back along the path the PING took. A connection is then formed.
- **Search — QUERY/QUERYHIT:** to find a file a node sends a **QUERY** (keyword) to its neighbours; each decrements the TTL and forwards it to *its* neighbours (except the one it came from). A node holding a matching file returns a **QUERYHIT** back along the reverse path, containing the **IP + port** so the requester can fetch the file by direct **HTTP** transfer.
- **Termination:** every message carries a **TTL** decremented at each hop; when TTL hits 0 the message stops. This bounds traffic but creates a **limited search horizon**.
- **NAT/firewall:** a peer behind NAT can't be connected *to* from outside. If the *requester* is behind NAT, it can still open the HTTP connection outward. If the *file-holder* is behind NAT, the requester sends a **PUSH** so the holder initiates the connection outward instead. (If *both* are behind NAT, direct transfer fails.)
- **Increasing TTL:** widens the horizon → reach more peers/files, **but** message volume grows fast (each extra hop multiplies flooding) → risk of network overload and wasted reach into irrelevant nodes.

A-GN2 — Gnutella 0.4 vs Gnutella 0.6

Layers: Application

Tags: `Gnutella-0.4` `Gnutella-0.6` `super-peer` `P2P` `flooding` **Book:** CB · **Seen:** bank Q13; 2020, 2022

0.6 introduces a **two-tier hierarchy**: **super-peers (ultra-peers)** carry the search/routing load; **leaf-nodes** just upload a list of their files to a super-peer and only receive a QUERY when they actually hold the match. Super-peers must have no firewall and enough bandwidth, uptime, RAM and CPU.

	0.4	0.6
Topology	flat, all peers equal	super-peers + leaves
Search load	flooded across everyone	concentrated on super-peers
Performance/scaling	worse	better
Anonymity	higher (only neighbours known)	**lower**

Exam trap / Hughes point: 0.6 buys performance but **sacrifices anonymity** — a few well-placed super-peers can monitor which leaf shares/downloads which file (easy copyright surveillance), undoing a key property of 0.4.

A-GN3 — Should Gnutella's transport be UDP or TCP? Trade-offs of switching

Layers: Application & Transport

Tags: `Gnutella-0.4` `TCP` `UDP` `flooding` **Book:** CB/OB · **Seen:** bank Q14; 2022, 2025-06-24

Two roles: **discovery** (PING/PONG/QUERY flooding) suits **UDP** — small, connectionless, broadcast-friendly, no handshake per hop. The **file transfer** uses **TCP/HTTP** because you need a reliable, ordered, complete download — you don't want missing chunks. - *Moving discovery to TCP:* every hop would need a connection → huge handshake overhead and latency in a flooding network — bad. - *Moving transfer to UDP:* you'd lose reliability/ordering and have to rebuild it yourself — bad for whole-file integrity.

So the natural answer is **UDP for control/discovery, TCP for transfer**.

A-GN4 — Spying on downloads in Gnutella 0.4 vs 0.6

Layers: Application

Tags: `Gnutella-0.4` `Gnutella-0.6` `super-peer` `P2P` **Book:** CB/OB · **Seen:** bank Q15; 2015-06-05

In **0.6**, register yourself as a **super-peer**: all routing and the file lists of your leaves pass through you, so you directly see who shares/requests what — monitoring is **easy**. In **0.4**, all peers are equal and each only sees its neighbours, so to observe meaningful traffic you must deploy **many** nodes spread across the network and passively log the QUERY/QUERYHIT flooding you happen to relay — much **harder** and only partial. (Contrast with the TOR surveillance question A-TOR2, which is about onion routing rather than P2P file lists.)

A-FTP — FTP: control vs data connection, the NAT problem, and passive mode

Layers: Application (interacts with Network/NAT)

Tags: `FTP` `active-mode` `passive-mode` `NAT` `NAT-traversal` **Book:** CB · **Seen:** bank Q16/Q17/Q18; 2017, 2018, 2023, 2024-open

- **Two connections (control port 21, data port 20):** separating them lets the client issue control commands (abort, list, new transfer) **while** a transfer is in progress. On one channel, a control command could get stuck behind buffered file data and arrive too late.
- **NAT problem:** in **active mode** the *server* opens the data connection back to a (random) **client-side** port. A client behind NAT/firewall has a private address and that inbound port is blocked → the data connection fails. (FTP also embeds IP addresses in its payload, which NAT doesn't rewrite — compounding the issue.)
- **Passive mode fix:** the **client** opens the data connection *to* the server (the server tells the client which IP/port to dial). Outbound connections pass through the client's NAT fine, so transfers work.

A-HTTP — HTTP 1.0 vs 1.1: advantages and when the gain is biggest

Layers: Application

Tags: `HTTP` `persistent-connection` `pipelining` `TCP` `connectionless` **Book:** CB · **Seen:** bank Q19; 2014, 2017-27, 2015-06-15

HTTP/1.0 opens a **new TCP connection per object** (image, script, …) — fine for simple pages but wasteful (handshake + slow-start cost each time) on modern pages with many objects. HTTP/1.1 uses **persistent connections** (reuse one TCP connection for many objects) and **pipelining** (request the next object as soon as it's known to be needed, without waiting for the previous response). **When biggest:** pages with **many embedded objects** — i.e. essentially all modern web pages. Side note: this is also why HTTP/1.0 is "connectionless/stateless" at the app layer while still using connection-oriented TCP (see A-L4).

A-TOR1 — TOR: what each node knows; minimum nodes; diminishing returns

Layers: Application

Tags: `TOR` `onion-routing` **Book:** CB · **Seen:** bank Q20; 2015-06-05

Onion routing wraps the message in layers of encryption, one per hop: - **Entry (guard) node:** knows **who the sender is**, but not the final destination or the content. - **Intermediate node(s):** know neither the origin nor the destination — only the previous and next hop. - **Exit node:** peels the **last** encryption layer and sees the **plaintext payload** (if no end-to-end encryption) and the destination, but not the original sender.

Minimum: 3 nodes (entry + ≥1 middle + exit). With fewer, one node could correlate *who* with *what*: an entry-that's-also-exit sees both source and plaintext; even a single relay between two endpoints leaks too much. **Diminishing returns:** more than ~3 (e.g. 10) adds little extra anonymity but a lot of latency — overkill.

A-TOR2 — Surveilling TOR with limited vs unlimited resources

Layers: Application

Tags: `TOR` `onion-routing` **Book:** OB · **Seen:** bank Q86; 2024-open

- **Limited (research project):** run a small number of **exit nodes** and log the plaintext leaving them (anything not end-to-end encrypted), plus traffic-pattern analysis. You see *content* but rarely the *originator* — a partial, opportunistic view.
- **Unlimited (state actor):** mount a **global traffic-correlation / timing attack** — observe both entry and exit points (or large fractions of the network) and correlate packet timing/volume to **de-anonymise** flows end to end. Running many guards and exits, plus monitoring core links, raises the chance you control or observe both ends of a circuit.

The key insight: TOR protects against a *local* adversary but is weak against an adversary who can observe **both ends** simultaneously.

A-P2P — Design a P2P chat app on Napster / Gnutella / BitTorrent

Layers: Application

Tags: `P2P` `Napster` `Gnutella-0.4` `Gnutella-0.6` `BitTorrent` `super-peer` **Book:** OB · **Seen:** bank Q85; 2024-open

Pick the right overlay **per phase** and justify it: 1. **Login / presence (who is online):** a **Napster-style central index** or a small set of **Gnutella-0.6 super-peers** — you need a reliable, queryable directory of online users; full flooding is wasteful for presence. 2. **Peer discovery / contact lookup: Gnutella 0.6** — super-peers route lookups efficiently while leaves stay light; flooding (0.4) doesn't scale to frequent lookups. 3. **Message / file exchange between two users: direct connection** (like Gnutella's HTTP transfer) for 1-to-1 chat; for sharing a large file to a **group**, use **BitTorrent** (swarming, chunked, parallel) so popular content scales.

Justify each choice by the phase's needs: directories want centralisation/super-peers; bulk distribution wants BitTorrent's swarm; 1-to-1 wants a direct link. Mention NAT traversal (PUSH/relay) as a practical wrinkle.

A-UDP3 — Name three protocols that use UDP and explain why they can't use TCP

Layers: Application & Transport

Tags: `UDP` `TCP` `DHCP` `DNS` `RTP` `broadcast` `anycast` `multicast` **Book:** OB · **Seen:** 2015-06-15, 2015-23 (not in 106 bank, but recurrent)

- **DHCP** — needs **broadcast** (client has no IP, server unknown); TCP is point-to-point.
- **DNS** — tiny one-shot lookups; TCP handshake overhead is wasteful, and UDP allows **anycast** redundancy.
- **RTP** — real-time media; needs low latency and tolerates loss, and uses **multicast/broadcast** for one-to-many streaming; TCP's retransmission would deliver data too late.

Common thread: each is **short, loss-tolerant, or one-to-many**, exactly where TCP's reliability/connection overhead hurts more than it helps.

Transport Layer

T-RTP1 — Goals of RTP and (≥3) key differences from TCP

Layers: Application & Transport

Tags: `RTP` `TCP` `UDP` `multicast` `congestion-control` **Book:** CB/OB · **Seen:** bank Q21; 2015, 2017, 2022

Goals: transport **real-time multimedia** and **multiplex/synchronise** several media streams (audio, video, subtitles) for smooth playback. The sending RTP library encodes streams into RTP packets over UDP; the receiver decodes and plays them, tolerating some loss.

Differences from TCP (pick 3): 1. **No retransmission** — a re-sent video frame arrives too late to be useful, so RTP doesn't bother (the app may interpolate). TCP always retransmits. 2. **Connectionless / runs on UDP** — no 3-way handshake; supports **multicast/broadcast** (TCP can't). 3. **No congestion control** — RTP (and UDP under it) don't throttle; TCP does. (This is also RTP's weakness — see T-RTP3.) 4. **Multiplexes multiple media streams** with timestamps for sync; TCP carries a single byte stream. 5. **Lives in user space**, shipped with the application, not in the OS kernel like TCP.

T-RTP2 — Explain the fields of the RTP header

Layers: Application & Transport

Tags: `RTP` **Book:** CB/OB · **Seen:** bank Q22; 2022-06-28

- **Version (V):** protocol version (2 bits).
- **Padding (P):** is the payload padded to a 4-byte boundary?
- **Extension (X):** is an extension header present?
- **CSRC count (CC):** number of contributing-source identifiers.
- **Marker (M):** marks a significant event, e.g. the start of a frame.
- **Payload type (PT):** which media encoding (codec) the payload uses.
- **Sequence number:** increments per packet → lets the receiver detect loss and reorder.
- **Timestamp:** sampling instant of the first byte → used for playout timing and inter-stream **synchronisation**.
- **SSRC (synchronisation source):** identifies the stream/source uniquely.
- **CSRC list:** the sources mixed into the payload (e.g. when a mixer combines streams).

T-RTP3 — RTP vs TCP: sequencing, video *download*, and high-volume traffic

Layers: Application & Transport

Tags: `RTP` `TCP` `congestion-control` `sliding-window` **Book:** OB · **Seen:** bank Q23/Q24/Q25 = Q88/Q89/Q90; 2017, 2023

- **Sequencing — why different:** both number their data, but RTP sequences at the **application layer** (sequence numbers in the RTP header, so the *app* can detect loss/reorder for media) while TCP sequences at the **transport layer** to guarantee in-order, reliable byte delivery. RTP only *detects* gaps; TCP *repairs* them.
- **Better for downloading a video file? No — TCP is better for a download**, because a file must arrive complete and in order, which TCP guarantees and RTP does not. RTP wins only for **live streaming**, where timeliness beats completeness. **Exam trap:** don't confuse "streaming" with "downloading".
- **High-volume RTP problem not seen with TCP:** RTP/UDP has **no congestion control**, so heavy RTP traffic can **congest the network** (packet loss, latency, quality collapse) with nothing throttling it. TCP would back off automatically.

T-HDR — Explain every field of the TCP header

Layers: Transport

Tags: `TCP-header` `TCP` `three-way-handshake` `sliding-window` `ECN` **Book:** CB · **Seen:** bank Q26; 2021, 2022

- **Source / Destination port (16 bits each):** the TSAP endpoints identifying the sending/receiving application.
- **Sequence number (32 bits):** byte-offset of the first data byte in this segment (if SYN=1, it's the *initial* sequence number).
- **Acknowledgement number (32 bits):** next byte expected = received sequence + bytes received; acknowledges all prior bytes.
- **Data offset / header length (4 bits):** size of the TCP header in 32-bit words (5–15 → 20–60 bytes), so the receiver knows where data starts (header is variable because of Options).
- **Reserved:** unused, set to 0 (future-proofing).
- **Flags: CWR/ECE** (ECN congestion signalling), **URG** (urgent pointer valid), **ACK** (ack field valid), **PSH** (deliver to app now, don't buffer), **RST** (reset/abort connection), **SYN** (open connection / sync sequence numbers), **FIN** (close connection). SYN+ACK form the **3-way handshake**.
- **Window (16 bits):** receiver's advertised free buffer → **flow control** (sliding window). 0 = "stop, I'm full".
- **Checksum (16 bits):** error detection over header + payload + IP pseudo-header.
- **Urgent pointer (16 bits):** with URG set, offset to the end of urgent data.
- **Options (0–40 bytes):** e.g. MSS, window scaling, SACK.

T-FLOW — Explain TCP flow control; the window probe; interaction with congestion control

Layers: Transport

Tags: `flow-control` `sliding-window` `window-probe` `congestion-control` `TCP` **Book:** CB · **Seen:** bank Q27/Q28/Q29; 2015, 2018, 2020, 2025-06-24

Flow control stops the *sender* overwhelming the *receiver's* buffer. The receiver piggybacks a **window advertisement** (free buffer in bytes) on its ACKs; the sender may have up to that many unacknowledged bytes outstanding. TCP **decouples** receiving ACKs from sending more data, so it doesn't stop-and-wait per segment. Walkthrough: sender writes 2 KB (SEQ 0); receiver ACKs with WIN=2048; sender sends another 2 KB; buffer full → ACK WIN=0 → sender **blocked**; app reads the buffer → ACK WIN=2048 → sender resumes.

When WIN = 0 the sender may still send only: **urgent data**, and a **window probe**. - **Window probe (define + when):** a tiny segment the sender transmits to re-query the receiver's window. It's sent because a window-*update* (the ACK saying "space freed") can be **lost**, which would otherwise **deadlock** both sides forever (sender waits for an update that never comes). The probe forces the receiver to re-advertise its window.

Interaction with congestion control (when the two windows differ): the sender is limited by the **minimum** of the **receiver (flow-control) window** and the **congestion window** — `usable = min(rwnd, cwnd)`. Flow control protects the *end host*; congestion control protects the *network*. Whichever is smaller binds.

T-FLOWGBN — Compare TCP flow control with Go-Back-N (and Selective Repeat)

Layers: Transport

Tags: `flow-control` `go-back-N` `selective-repeat` `sliding-window` `SACK` `NACK` **Book:** OB · **Seen:** bank Q30; 2017, 2015-23

	TCP	Go-Back-N
Sliding window	yes	yes
ACKs	cumulative + (with SACK) selective	cumulative
Retransmission	only the missing bytes (with SACK)	resends the **whole window** from the lost packet
Receiver buffering	yes (stores out-of-order)	typically discards out-of-order
Congestion control	yes	no

Similarities: both are sliding-window protocols allowing several unacknowledged packets in flight, both aim for reliable delivery, both can suffer under loss. **Difference vs Selective Repeat:** Selective Repeat buffers and retransmits *only* the specific lost segment (like TCP+SACK) and requires window ≤ ½ the sequence-number range (see T-WINCONSTRAINT); Go-Back-N retransmits everything after the gap.

T-CONGMECH — List the ways TCP copes with varying network conditions

Layers: Transport

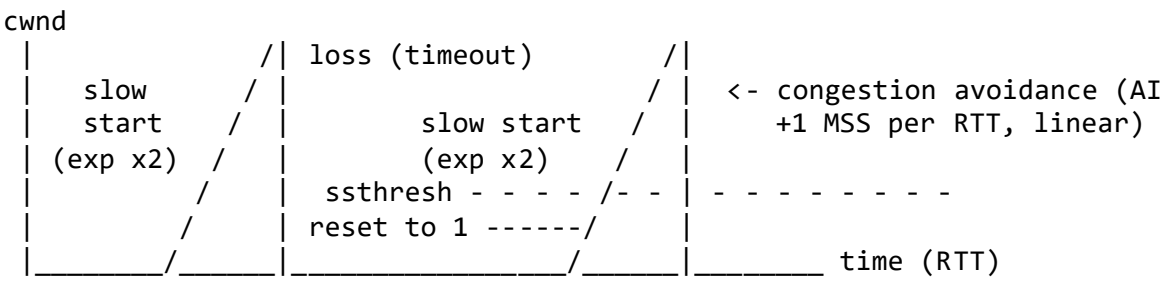
Tags: `congestion-control` `flow-control` `sliding-window` `SACK` `slow-start` `AIMD` `TCP`
Book: CB/OB · **Seen:** bank Q31 = Q91; 2017-27, 2023

- **Congestion control** (Tahoe/Reno/CUBIC): slow start then AIMD, reacting to loss/delay to find a safe rate.
- **Sliding window:** bounds in-flight unacknowledged data; size adapts to congestion + receiver capacity.
- **Retransmission + adaptive timeout (RTO):** lost/unacked segments are re-sent; RTO is tuned from measured RTT (smoothed RTT + variance).
- **Selective ACK (SACK):** receiver acks non-contiguous blocks so the sender resends only what's missing.
- **Flow control:** advertised receive window stops overrunning the receiver.
- **Window scaling:** Options-field extension beyond the 16-bit window for high-bandwidth links.

T-TAHOE — Draw and annotate TCP Tahoe's congestion window

Layers: Transport

Tags: `TCP-Tahoe` `slow-start` `ssthresh` `AIMD` `congestion-control` **Book:** CB · **Seen:** bank Q32; 2024-closed



Phases: 1. **Slow start:** cwnd starts at 1 MSS and **doubles every RTT** (exponential — the name is misleading; it ramps up *fast*) until it reaches **ssthresh** (or loss). 2. **Congestion avoidance (AIMD):** above ssthresh, cwnd grows **linearly** (+1 MSS/RTT) — gentle probing near capacity. 3. **Loss (timeout):** Tahoe sets cwnd **all the way back to 1 MSS** and **ssthresh = ½ of the window at loss**, then re-enters slow start. It drops to 1 because, without NACK/SACK, it doesn't know which packet was lost, so it "drains the pipe". (Later NACK/SACK fixed this.)

T-RENO — TCP Tahoe vs TCP Reno

Layers: Transport

Tags: `TCP-Tahoe` `TCP-Reno` `fast-recovery` `SACK` `congestion-control` **Book:** CB · **Seen:** bank Q33; 2022

Both: slow start → AIMD. The difference is the **loss reaction**: - **Tahoe:** any loss → cwnd back to **1** → full slow start again (slow recovery). - **Reno:** adds **Fast Recovery** — on loss (triple-duplicate-ACK) it **halves** cwnd (down to ~½, not 1) and continues in congestion avoidance, so it recovers far faster. Reno only does full slow start at the very start.

Beyond Reno: SACK (selective ACK in Options, backwards-compatible — non-SACK hosts ignore it) for precise recovery, and **ECN** (ECE/CWR bits). **Hughes point:** ECN is *less deployed* than SACK because ECN must be implemented on the **routers** as well as both endpoints, whereas **RED** runs on routers alone (and helps the router itself), so RED is what actually got deployed.

T-QUIC — QUIC: design overview, differences from TCP, congestion control vs Reno/Tahoe

Layers: Transport

Tags: `QUIC` `TCP` `UDP` `congestion-control` `SACK` `NAT` **Book:** CB · **Seen:** bank Q34 = Q44; 2024-closed, 2025-06-10

QUIC is a modern transport built **in user space on top of UDP**. Why UDP underneath: middleboxes/NAT understand UDP but not QUIC, and user-space avoids OS-kernel **ossification** (TCP changes need OS updates everywhere). Key features: - **Built-in encryption (TLS)** — QUIC+TLS sets up faster than TCP+TLS. - **Low-latency connection establishment** (0/1-RTT). - **Reliable, in-order, multiplexed streams** *with* retransmission — but **multiple independent streams** avoid TCP's **head-of-line blocking** (one lost packet doesn't stall every stream). - **Connection migration:** keep the connection alive across a change of **IP/port** (e.g. WiFi→4G). Sending rate is limited until the new address is validated (PATH_CHALLENGE/RESPONSE), and congestion state is reset. - Frame-based; 62-bit sequence numbers (never wrap); ACKs carry received-byte ranges (SACK-like, across streams).

Differences from TCP: encryption mandatory; per-stream multiplexing (no HoL blocking); user-space/UDP; connection migration; faster setup. **Congestion control — yes, QUIC has it**, typically based on **CUBIC** (the same family modern Linux TCP uses), conceptually like TCP Reno/Tahoe (slow start + congestion avoidance + loss response) but decoupled from kernel TCP and per-connection. **Exam trap:** QUIC is **not** a drop-in RTP replacement — it currently has **no unreliable mode**, so it can't serve loss-tolerant real-time media the way RTP/UDP does.

T-ACKCLOCK — How does an ACK clock work?

Layers: Transport

Tags: `ACK-clock` `sliding-window` `congestion-control` `TCP` **Book:** CB/OB · **Seen:** bank Q35 = Q92; 2017-27, 2023

Picture a fast sender→router link and a slow router→receiver link. A burst of packets queues at the router and crosses the slow link one at a time; the receiver's ACKs are therefore **spaced out to match the slow link's rate**, and that spacing is preserved on the way back. The sender uses the **arrival of ACKs to *clock* new transmissions** — inject one new packet per returning ACK. Result: the sender naturally sends at the **bottleneck rate**, keeping the pipe full **without** building queues or congesting routers. It is "self-clocking" feedback.

T-NACK — What is a NACK and when is it used?

Layers: Transport

Tags: `NACK` `selective-repeat` `sliding-window` **Book:** CB · **Seen:** bank Q36; 2020

A **Negative ACKnowledgement** explicitly tells the sender the **sequence number of a missing or corrupted/out-of-order segment**, requesting just that one be re-sent. A receiver sends it when it gets a packet with a sequence number **higher than expected** (so the ones in between were lost) — signalling "this arrived, but the earlier ones didn't". It is associated with **Selective Repeat**, and it lets you avoid retransmitting correctly-received packets (unlike Go-Back-N).

T-WINCONSTRAINT — Constraint on window size from the sequence-number range

Layers: Transport

Tags: `sliding-window` `selective-repeat` `NACK` **Book:** CB · **Seen:** bank Q37; 2020

For Selective Repeat, the **window size must be ≤ ½ the sequence-number range**. If it's larger, then when ACKs are lost the sequence numbers at the receiver can **wrap around** and overlap the sender's retransmissions, so the receiver can't tell a *retransmitted old* packet from a *new* one with the same number — and it accepts a duplicate as new. Half the range guarantees no ambiguity.

T-TINYGRAM — Tinygram & Silly Window Syndrome: Nagle, Clarke, delayed ACKs

Layers: Transport

Tags: `tinygram` `silly-window` `Nagle` `Clarke` `delayed-ACK` `TCP` `sliding-window` **Book:** CB · **Seen:** bank Q39/Q40/Q41 = Q93; 2018, 2020, 2021, 2025-06-10

Tinygram syndrome = catastrophic *sender-side* overhead from sending 1 byte at a time (e.g. Telnet keystrokes): worst case **120 bytes for 1 byte of data** — transmission (IP 20 + TCP 20 + 1), the ACK (20+20), and the window update (20+20). Fixes: - **Delayed ACKs:** the receiver waits (until a timeout) before ACKing, hoping to **piggyback** the ACK on return data and to batch ACKs. - **Nagle's algorithm:** while there is **unacknowledged** data outstanding, **buffer** new small output and send it only once you have a **full segment** (or the outstanding data is ACKed). Downside: added latency — bad for games/real-time, where you disable it.

Silly Window Syndrome = the *receiver-side* mirror: the app reads 1 byte at a time, so the receiver keeps advertising tiny windows, and the sender keeps sending tiny segments → huge overhead. Fix: - **Clarke's algorithm:** the receiver **delays window updates** until it can accept a **full MSS** or its buffer is **half empty**. Complementary to Nagle (sender won't send small, receiver won't ask for small).

Do Nagle + delayed ACKs play well together? No. Nagle waits for an ACK before sending more; delayed ACKs wait before ACKing — so each side waits for the other → **added latency or deadlock** (timeout after timeout). For interactive/low-latency apps, disable Nagle.

T-ECN — ECN: operation; advantages/disadvantages vs choke packets and vs RED

Layers: Transport & Network

Tags: `ECN` `choke-packets` `RED` `congestion-control` `TCP-header` **Book:** CB · **Seen:** bank Q42/Q43; 2018, 2024-closed, 2025-06-10

Operation: a router nearing congestion **marks** passing packets by setting the **ECN** bits instead of dropping them; the receiver sees the mark and tells the sender (via the **ECE** flag) to **slow down**, and the sender acknowledges with **CWR**. It needs the **ECN bits in the IP/TCP header** and support at **routers *and* both endpoints**.

vs **Choke packets** (router sends an explicit packet to the source): ECN adds **no extra traffic** (it reuses existing packets), so it doesn't worsen an already-congested network; choke packets *add* load — useless once the network is saturated. **Disadvantage of ECN:** it is **slower to take effect** (the signal travels to the receiver and back), and it must be implemented everywhere (routers + hosts), unlike a choke packet that only the sender must understand.

vs **RED** (router probabilistically *drops* packets early): see the full comparison in **N-CONG**. In short, ECN signals without dropping (lower loss) but needs end-host support; RED needs only the router but relies on packet loss as the signal.

T-UDPSERV — What services does UDP offer, and when are they useful?

Layers: Transport

Tags: `UDP` `broadcast` `multicast` `RTP` **Book:** OB · **Seen:** bank Q94; 2014, 2015-06-05, 2023

UDP offers **connectionless**, **unreliable** delivery with **low overhead** plus **broadcast/multicast** support. Useful when **low latency and speed beat guaranteed delivery**: live streaming, gaming, VoIP, real-time monitoring, and small frequent or one-to-many messages (DNS, DHCP). Apps needing reliability must add ordering/loss handling themselves (as **RTP** does on top of UDP). It works for DHCP precisely because DHCP needs broadcasting.

T-RETRANS — Why is retransmission good at the Transport layer? Advantages of Data-Link retransmission

Layers: Transport & Data Link

Tags: `TCP` `congestion-control` `Ethernet` `802.11` **Book:** OB · **Seen:** bank Q95; 2015-06-05

Transport-layer retransmission (TCP): gives **end-to-end** reliability across the *whole* path — it detects and repairs loss/corruption regardless of how many heterogeneous links lie in between, and ties into congestion control. It's the only place that can guarantee the *complete* end-to-end delivery. **Data-Link retransmission (e.g. 802.11 ACKs):** repairs loss **locally, hop-by-hop**, with **much lower latency** (you don't wait a full RTT to the far endpoint) and avoids a lossy wireless hop poisoning the end-to-end view. Its limit: it's only local — it can't guarantee end-to-end delivery. The two are complementary; local recovery on a flaky link prevents TCP from misreading link loss as congestion.

> **Leaky/token bucket** (traffic shaping) appeared only in a 2015 paper and is likely de-emphasised now: a *leaky bucket* drains at a fixed rate **R** (smooths bursts into a constant stream); a *token bucket* of size **B** lets you send a burst of up to B then continue at rate R (allows controlled bursts). Tags: `leaky-bucket`.

Network Layer

N-CONG — Compare choke packets, ECN and RED; describe RED

Layers: Network & Transport

Tags: `choke-packets` `ECN` `RED` `congestion-control` **Book:** CB/OB · **Seen:** bank Q45/Q46; 2014, 2015, 2023

	Choke packets	**ECN**	**RED**
Mechanism	router sends an explicit packet to the	router marks bits in passing	router probabilistically **drops** packets early

	Choke packets	ECN	RED
	source	packets	
Speed (signal→reaction)	fast	relatively fast	**slower** (sender must first notice the loss)
Efficiency	lower (extra packets, retransmits)	higher	**highest**
Extra load on network	**yes** (adds traffic)	none	none
Needs end-host support	sender only	routers **+** both endpoints	routers only

RED (Random Early Detection): the router watches its **average queue length**; once it exceeds a threshold it starts dropping packets **probabilistically**, with drop probability **rising as the queue grows**. Because heavy senders have more packets in the queue, they **lose more and back off first** — congestion is signalled fairly with **minimal overhead**, *before* the buffer overflows. **Hughes point:** RED is the one actually deployed on routers (it helps the router itself); ECN lags because it also needs host support.

N-OSPFBGP — OSPF vs BGP; why OSPF uses UDP and BGP uses TCP

Layers: Network & Transport

Tags: `OSPF` `BGP` `autonomous-system` `link-state` `UDP` `TCP` **Book:** CB/OB · **Seen:** bank Q47; 2015-23, 2022-06-28

	OSPF	BGP
Scope	**inside** an AS (corporate/campus)	**between** ASes (ISPs)
Algorithm	link-state (Dijkstra), full network knowledge	path-vector with **policy**
Knowledge	every router knows the whole area	border routers know reachability + policy
Scale	not for country-scale	designed for the global Internet
Policy	naive shortest-path	rules on cost/bandwidth/which ASes to avoid

Why OSPF "uses UDP": it **floods** link-state advertisements and needs lightweight, broadcast-style, connectionless delivery — a per-neighbour TCP session would be heavy and pointless for flooding. **Why BGP uses TCP:** inter-AS sessions are **long-lived** and must be **reliable** (no lost/reordered routing updates between ISPs), and TCP gives that reliability + ordered delivery + flow control over a stable peering. **Factual aside (don't lose marks, but know it):** real OSPF runs **directly on IP (protocol 89)**, not literally on UDP — it's connectionless *like* UDP, which is the spirit the course means.

N-AODVOSPF — When is AODV preferable to OSPF?

Layers: Network

Tags: `AODV` `OSPF` `distance-vector` `link-state` **Book:** CB · **Seen:** bank Q48; 2020, 2022

AODV (Ad hoc On-demand Distance Vector) is **reactive**: it discovers a route *only when* a datagram needs to be sent, so it suits **highly dynamic networks** where nodes move and links appear/disappear (mobile ad-hoc, event WiFi, sensor meshes) — it avoids constantly recomputing routes that may never be used. **OSPF** is **proactive**: it pre-computes and maintains full routes, ideal for **stable** corporate/campus networks where the topology rarely changes, so there's no per-packet route-setup delay. **Hughes one-liner:** *reactive (AODV) for dynamic, proactive (OSPF) for static.*

N-DVR — Distance Vector Routing & the count-to-infinity problem (incl. AODV's fix)

Layers: Network

Tags: `distance-vector` `count-to-infinity` `split-horizon` `AODV` `OSPF` **Book:** CB · **Seen:** bank Q49/Q50; 2015, 2020, 2022

Distance Vector (Bellman-Ford, e.g. RIP): each router keeps a table of (destination → distance, next-hop) and periodically shares it with neighbours; each updates its own table from neighbours' advertised distances + 1.

Count to infinity: "good news travels fast, bad news travels slowly." When a node/link **fails**, the disconnected node should be removed, but before the bad news propagates, a neighbour may still advertise an **old route** to it (e.g. cost 2). A router unknowingly adopts that route (cost = advertised + 1), re-advertises it, the other side adds 1 again, and the distance **climbs slowly toward infinity** — because each round can only increase a path by 1. Meanwhile routers may form **loops/suboptimal paths**. *Example* (line E–A–B–C–D, E fails): A=1→3→3→5→5…, B=2→2→4→4→6… each exchange creeps upward.

Mitigations: split horizon (don't advertise a route back to the neighbour you learned it from), **poison reverse**, a **maximum metric** that *defines* infinity (RIP uses 16), and **hold-down timers**. (Link-state protocols like OSPF avoid it entirely by sharing full topology.)

How AODV handles it: AODV is on-demand and uses **destination sequence numbers** + **HELLO** messages. If a node stops hearing HELLOs from an active neighbour it declares the link down, **purges** the route, and floods a **route-error (RERR)** to everyone using that route — recursively — so stale routes are removed instead of being counted up. It still has long timeouts but **sidesteps** count-to-infinity.

N-MESH — Routing for a constrained Zigbee/BLE mesh with an IPv6 gateway

Layers: Network

Tags: `AODV` `RPL` `Zigbee` `BLE` `IoT` `IPv6` `distance-vector` **Book:** OB · **Seen:** bank Q101; 2025-06-24

The devices are **mobile-ish, energy- and resource-constrained**, multi-hop, talking to one **IPv6 gateway**. The best fit from the lectures is a **reactive, low-overhead protocol — AODV**: it only builds routes on demand (no constant proactive flooding to drain batteries) and copes with a changing topology. **Modifications to make it more efficient:** - **Energy-aware metric** (prefer routes through nodes with more battery, not just fewest hops). - **Reduce control overhead** (fewer/aggregated HELLOs, longer route lifetimes). - Exploit the gateway as a **sink**: build a destination-oriented structure toward it (this is exactly what the real-world IoT protocol **RPL** does — a DODAG rooted at the gateway) and **aggregate** sensor data on the way up.

N-IPMC — Packet-Tracer routing MCQ (dest/src IP, default gateway, ping failure, broadcast scope)

Layers: Network (+ Data Link for MAC)

Tags: `IP-addressing` `default-gateway` `broadcast-address` `ARP` `IPv4` **Book:** OB · **Seen:** bank Q51; Toledo, 2021, 2023

Topology: PC-A/PC-B → S1 → R1 — R2 → S2 → PC-C/PC-D. - **Destination IP when A→C (as it leaves R1):** the **final destination's IP** = PC-C (e.g. 10.20.5.5). IPs are **end-to-end and don't change**; only MAC addresses change per hop. - **Source IP (as it leaves R1):** still **PC-A's IP** (e.g. 10.0.0.7) — unchanged. - **Default gateway of PC-A:** the router interface's **IP** (e.g. 10.0.0.1). **Exam trap:** it is *not the switch* (switches/hubs/repeaters are layer-1/2 and have **no IP**). "R1" is informally the gateway *node*, but the precise answer is the **IP address** of R1's interface on A's subnet. - **Ping to R1's far interface fails (choose all causes):** *plausible* — (a) the request reaches R1 but the **reply can't be routed back**, and (c) **R1 lacks a valid route** to the source subnet. *Not* a cause: "R1 isn't C's default gateway." - **Broadcast by A to `10.0.255.255/16` — who receives it?** Only hosts on **A's own /16 LAN** (PC-A, PC-B, R1's near interface). **Exam trap:** broadcasts **do not cross routers**, so PC-C/PC-D on the other LAN get nothing.

N-SUBNET — IP addressing & subnetting: the method, with every exam variant solved

Layers: Network

Tags: `IP-addressing` `subnetting` `CIDR` `subnet-mask` `broadcast-address` `IPv4` **Book:** OB · **Seen:** bank Q52/Q53; 2017, 2020, 2022, 2025-06-10, 2025-06-24

Method (works for any mask): 1. **CIDR /n** = first *n* bits are network; the rest are host. `/28` → mask `255.255.255.240`, `/20` → `255.255.240.0`, `/26` → `255.255.255.192`. 2. **Network**

address = IP **AND** mask. **Broadcast** = network with **all host bits = 1**. **First host** = network + 1. **Last host** = broadcast − 1. **#usable hosts** = 2^(host bits) − 2. 3. **Same subnet?** compute (IP **AND** mask) for both addresses with the *same* mask; equal ⇒ same subnet. 4. **Private (RFC 1918):** `10/8`, `172.16/12`, `192.168/16`.

Worked answers from the papers: - **172.16.0.1** → IPv4 **private** address (in 172.16/12). - **172.29.206.0, mask 255.255.254.0 (/23):** 9 host bits → broadcast **172.29.207.255**. - **172.22.7.16/28:** block size 16, 14 hosts → **first host 172.22.7.17**, **last host 172.22.7.30** (network .16, broadcast .31). **Trap:** not .16 / .31. - **/26 (255.255.255.192) generic X.X.X.X:** last 6 bits are host → broadcast = network OR `00111111` (i.e. `.??111111`). - **255.255.240.0 (/20):** 12 host bits → $\mathbf{2^{12} - 2 = 4094}$ usable hosts. - **192.168.242.5/20:** 242 AND 240 = 240 → network **192.168.240.0**, **broadcast 192.168.255.255**. - **172.23.193.64, mask 255.255.192.0 (/18):** 193 AND 192 = 192 → network **172.23.192.0**, **broadcast 172.23.255.255**. *(One 2025 transcription wrote the mask as 255.192.0.0 (/10) — apply whatever mask is actually printed; the method is identical.)* - **Is 162.16.168.30 in 152.16.168.196/28? No. Exam trap:** they differ in the very **first octet** (152 vs 162), so they're in completely different networks — the /28 is a red herring; you never even reach the host bits.

N-V6STRUCT — Structure of an IPv6 address

Layers: Network

Tags: `IPv6` `IP-addressing` **Book:** OB · **Seen:** bank Q54; 2014, 2015-06-15

128 bits, written as **eight 16-bit "hextets"** in hex separated by colons. Leading zeros in a hextet may be dropped, and **one** run of all-zero hextets may be replaced by `::` (only once, else it's ambiguous). It splits into a **network prefix** + an **interface identifier** — commonly **64 bits routing/prefix + 64 bits host**. Address types: **unicast, multicast, anycast**.

N-V6SHORT — Shorten an IPv6 address (method + worked)

Layers: Network

Tags: `IPv6` `IP-addressing` **Book:** OB · **Seen:** bank Q55; 2014, 2015-06-15, 2025-06-24

Steps: (1) replace the **longest run of consecutive all-zero hextets** with `::` (once); (2) drop **leading zeros** in each remaining hextet. - `2041:0000:140f:0000:0000:0000:875b:131b` → **`2041:0:140f::875b:131b`** - `2a02:0000:0000:0000:0000:0a3e:df3f:4b19` → **`2a02::a3e:df3f:4b19`**

N-V6SLAAC — IPv6 stateless autoconfiguration: security, privacy, vs DHCPv6

Layers: Network

Tags: `SLAAC` `autoconfiguration` `EUI-64` `DHCPv6` `IPv6` **Book:** OB · **Seen:** bank Q56/Q57/Q58; 2017-27, 2015-06-05, 2025-06-24

SLAAC builds the interface ID from the NIC's **MAC address (EUI-64)**. Problems: - **Privacy:** the address **embeds the MAC**, so it's **stable and traceable** across networks — you can track a user/device. (An address with `ff:fe` in the middle is the EUI-64 fingerprint.) - **Security:** addresses are **predictable/enumerable** → easier scanning and targeting; SLAAC implies **no gateway/firewall in the middle by default**, so inbound connections aren't filtered → easier remote attacks and **rogue-device onboarding**.

Why DHCPv6 reveals less / is safer: a DHCPv6 server **assigns** addresses centrally (it can randomise them, doesn't derive from the MAC), keeps **logs/control**, and fits a managed network with filtering. SLAAC leaks more because the host derives its own address from hardware identifiers with no central mediation. **2025 sub-questions:** SLAAC suits **always-on / IoT** devices that must autoconfigure with no server; an EUI-64-looking address is **likely** SLAAC. (RFC 4941 *privacy extensions* randomise the interface ID to mitigate the tracking problem.)

N-V6CHK — Why is there no checksum in the IPv6 header?

Layers: Network

Tags: `IPv6` `checksum` `ICMP` **Book:** OB · **Seen:** bank Q59; 2014, 2015-06-15

Error detection already happens **below** (link layer: Ethernet/802.11 frame checks) and **above** (transport: TCP/UDP checksums, which cover an IP pseudo-header). A separate IPv6 checksum would be **redundant**. Dropping it makes **per-hop processing faster** — crucially, a router decrements the hop limit on **every** packet, and with a header checksum it would have to **recompute** that checksum at every hop. Removing it streamlines forwarding.

N-FRAG — MTU discovery, IPv4 fragmentation & ICMP (why incompatible; fix; segmentation vs fragmentation)

Layers: Network (segmentation note: Transport)

Tags: `MTU` `path-MTU-discovery` `fragmentation` `segmentation` `ICMP` `DF-bit` `IPv4` **Book:** OB · **Seen:** bank Q61/Q62 = Q96/Q97; 2017, 2018, 2023, 2024-open

- **Fragmentation (network layer):** a router splits a packet too big for a link's **MTU** into fragments, reassembled at the destination.
- **Segmentation (transport layer):** TCP slices the byte stream into **segments ≤ MSS** before handing them down. (They sit at different layers and serve different purposes — that's the "difference in approaches".)
- **MTU / Path-MTU discovery:** the sender sets the **DF (Don't Fragment)** bit and sends a large packet; a router that can't forward it **drops it and returns an ICMP "fragmentation needed"** message carrying that link's MTU; the sender shrinks and retries until it gets through → it learns the **path MTU**. **ICMP is essential** — it's the feedback channel that makes discovery work.

- **Why MTU discovery and fragmentation are mutually incompatible:** discovery **assumes fragmentation is off** (DF set, fragmentation undesirable). If fragmentation is **allowed** (DF clear), routers silently fragment instead of sending the ICMP, so the sender **never learns** the real MTU — discovery is defeated.
- **Fix + efficiency:** have a router send an ICMP **each time it fragments** (reporting the fragmentation + the MTU), so the sender learns the path MTU in essentially one pass and then sends right-sized packets. Cost: a one-time overhead at routers, then a speed-up (no further fragmentation). Caveat: arguably pointless, since simply **setting the DF bit** already achieves clean MTU discovery.
- **Do all packets traverse the network layer identically? No** — packet-switching can send packets along **different routes** (varying delay), and **DiffServ/QoS** classes (gold/silver/bronze) prioritise some traffic and drop large bursts first under congestion.

N-NAT — NAT's problems at the application layer + two traversal techniques

Layers: Network & Application

Tags: `NAT` `NAT-traversal` `FTP` `passive-mode` `PUSH` **Book:** OB · **Seen:** bank Q63 = Q87; 2023, 2025-06-10, 2025-06-24

NAT rewrites IP/port at the boundary, but it **doesn't rewrite IP addresses embedded inside application payloads**, and it blocks **inbound** connections to private hosts. So protocols that carry their own addresses break: **FTP active mode** (the payload tells the server a private client IP/port to dial back), **SIP/VoIP**, etc. → the data/return connection fails.

Two techniques applications use to cope: 1. **Client-initiated connections** — e.g. **FTP passive mode** (client dials the server, so outbound through NAT is fine); Gnutella's **PUSH** (ask the file-holder to connect *out* to the requester). 2. **Relays / hole punching / helpers — STUN/TURN** (discover the public mapping or relay through a server), **ALG** (NAT understands and rewrites known payloads), **UPnP** port mapping.

N-IOT — IoT vs a full IP stack: problematic features, IPv4 vs IPv6, IP vs non-IP

Layers: Network

Tags: `IoT` `IPv4` `IPv6` `IP-addressing` **Book:** OB · **Seen:** bank Q60/Q100; 2018, 2023

IoT features problematic for IP: - **Scalability / addressing:** billions of devices vs IPv4's exhausted address space. - **Power & resource constraints:** the full IP/TCP stack is heavy on CPU/memory/energy for tiny battery devices. - **Bandwidth/overhead:** IP + TCP headers dwarf the tiny sensor payloads.

IPv4 vs IPv6 for IoT: IPv4 = mature, compatible, but **exhausted** addresses and limited security; IPv6 = **huge address space + stateless autoconfig** (built for always-on devices) but adoption/upgrade cost. **Hughes leans IPv6** for IoT. **IP vs non-IP:** IP gives **interoperability/standardisation** with the existing Internet; **non-IP** (optimised stack +

link-layer addresses, e.g. Zigbee/BLE) gives **resource efficiency** for tailored deployments. The right choice depends on scale, constraints, and how much Internet interoperability you need. (IoT devices typically live at the physical/data-link/network layers near a gateway.)

N-ADDR — Compare addressing at Network, Data Link and Application layers

Layers: Network, Data Link & Application

Tags: `IP-addressing` `NSAP` `TSAP` `port` `ARP` **Book:** OB · **Seen:** bank Q102; 2015-06-05

Layer	Address	Scope / purpose
Network	**IP / NSAP**	logical, **end-to-end routing** between networks
Data Link	**MAC**	physical, **next-hop delivery within one link/segment**
Application	**port / TSAP**	identifies the **process/application** on a host (demultiplexing)

Why all three are needed: they work at **different scopes**. IP gets the packet across the internet (stays constant end-to-end); MAC delivers it across each individual hop (**changes per hop**, resolved via **ARP**); the port lets one host run many apps over a single IP (**why we have TSAPs** — multiplexing). Drop any one and delivery breaks: no IP = can't route between networks; no MAC = can't deliver on the wire; no port = can't tell which app gets the data.

N-TTL — Why no TTL in Ethernet, and can you move it to TCP/UDP?

Layers: Network & Data Link & Transport

Tags: `TTL-field` `Ethernet` `IPv4` **Book:** OB · **Seen:** 2015-23 (not in 106 bank, but recurrent)

- **Why Ethernet has no TTL:** Ethernet is **data-link/single-hop**; it has no concept of multi-hop routing or loops to protect against, so a hop-count is meaningless there. (Loop prevention in switched LANs is handled by spanning tree, not a TTL.)
- **Replace the IP TTL with a field in the TCP/UDP header? No.** TCP/UDP are **transport/end-to-end** and have **no notion of the intermediate routers/hops**, so they can't decrement per hop. TTL must live at the **network layer**, where routing (and thus loop risk) happens.

N-DESIGN — Open-book network-design exercise (addresses, PDUs, routing tables, ARP)

Layers: Network (+ Data Link, Transport)

Tags: `IP-addressing` `NAT` `routing-table` `ARP` `NSAP` `TSAP` **Book:** OB · **Seen:** 2015-06-15, 2015-23 (large design question)

This tests whether you can apply everything. Approach: - **Assign addresses:** private (RFC 1918) **inside**, public **outside**; the **NAT** box translates between them. - **PDU contents at each tap point** (e.g. Client1 → Google, points A→B→C→D): src/dst **IP** (NSAP), src/dst **MAC** (changes every hop), src/dst **port** (TSAP), + payload. *Before* NAT: **private** client IP/port;

after NAT (toward the Internet): **public** client IP/port. The application endpoints' **IPs stay constant**, **MACs change per hop**, NAT swaps the client-side IP/port at the boundary, and the final hop carries Google's server MAC. - **Routing tables:** each entry = **destination network → next-hop gateway + outgoing interface**. - **ARP cache: IP→MAC** mappings the device has learned for next-hops on its local segment (filled in as ARP resolves them). - **Replace the NAT box with a router given one public IP?** Only if that router can also do the **address translation** — sharing one public IP among many hosts *requires* NAT functionality, so a plain router won't do.

Data Link & Physical Layer

D-HIDDEN — Hidden & exposed terminal problems; MAC solutions; does CSMA help; channel hopping

Layers: Data Link

Tags: `hidden-terminal` `exposed-terminal` `CSMA/CA` `RTS/CTS` `MACA` `channel-hopping` `802.11` `MAC-protocol` **Book:** CB · **Seen:** bank Q64; 2017, 2014

- **Hidden terminal:** A and C both want to send to B but are **out of radio range of each other**. Each senses the channel idle (it can't hear the other), both transmit, and their signals **collide at B** → B receives garbage. Carrier sensing fails because the sender can't detect a transmitter it can't hear.
- **Exposed terminal:** B is transmitting to A; C wants to transmit to D. C hears B and **politely backs off**, even though C→D wouldn't actually collide at the intended receivers. So **no collision would occur, but a valid transmission is needlessly suppressed** → wasted capacity.

How MAC protocols address the *hidden* terminal (give 2): 1. **RTS/CTS handshake (MACA/MACAW, used in 802.11):** the sender broadcasts a short **Request-To-Send**; the receiver replies **Clear-To-Send**. Every node that hears the **CTS** (including the hidden node) knows to stay silent for the announced duration → the hidden node is informed even though it can't hear the sender. 2. **TSMP / scheduled access:** a global **mesh schedule** assigns transmit slots, so out-of-range nodes never clash (it has the network-wide view that plain carrier sensing lacks).

Does CSMA help? Not really. Plain CSMA only checks if the *local* channel is busy; it has **no notion of activity outside radio range**, so it can't fix the hidden terminal, and for the *exposed* terminal CSMA actually *causes* the problem (it refuses to send on a "busy" channel). *p*-persistent CSMA can statistically reduce hidden-terminal collisions but never eliminate them.

Channel hopping (benefit): transmit on a **pseudo-random sequence of frequencies** so that **interference** (microwaves, motors, other senders, shielding) — which is usually

narrow-band — only hits a fraction of the hops. It minimises the *influence* of interference you can't otherwise control.

D-ALOHA-80211 — Can Pure ALOHA be implemented on 802.11?

Layers: Data Link & Physical

Tags: `ALOHA` `802.11` `CSMA/CA` `WiFi` **Book:** CB · **Seen:** bank Q65; 2014, 2024-closed

The **802.11 physical layer** (radio) *can* carry a Pure-ALOHA scheme — ALOHA just transmits whenever it has data and re-sends after a random delay if it collides, which the radio hardware supports. But it would be a **bad fit**: 802.11 deliberately uses **CSMA/CA** (sense first, avoid collisions, RTS/CTS) precisely because raw ALOHA's blind transmissions waste a shared radio channel and ignore hidden terminals. So: physically possible on the same medium, but you'd throw away the collision-avoidance that makes WiFi usable. The underlying principle is shared (radio, same interference issues); CSMA/CA is the refinement — "**back off aggressively, send gently**".

D-ALOHA-PERF — Describe Pure ALOHA; compare its performance to Slotted ALOHA

Layers: Data Link

Tags: `ALOHA` `slotted-ALOHA` `MAC-protocol` **Book:** CB · **Seen:** bank Q66; 2017

Pure ALOHA: a station transmits a frame **whenever it is ready** (no carrier sense, no slots). If two frames overlap *at all*, both are destroyed; each sender waits a **random** time and retransmits. A frame is vulnerable for **two frame-times** (a collision can start just before or just during it). **Slotted ALOHA:** time is divided into **slots**; a station may only start at a **slot boundary**. This halves the vulnerable window to **one** slot.

	Pure ALOHA	Slotted ALOHA
When you may send	anytime	only at slot start
Vulnerable period	2 frame-times	1 slot
Max channel efficiency	**≈ 1/(2e) ≈ 18%**	**≈ 1/e ≈ 37%**
Needs synchronisation	no	yes (slot timing)

So slotting **doubles** throughput at the cost of requiring time synchronisation across stations.

D-POWER — Two 802.11 power-saving strategies + a use case each

Layers: Data Link

Tags: `power-saving` `beacon-frame` `TIM` `PS-Poll` `APSD` `802.11` `duty-cycle` **Book:** CB · **Seen:** bank Q67; 2020, 2022

1. **Beacon + TIM / PS-Poll (scheduled wake-up):** the AP periodically sends a **beacon** containing a **Traffic Indication Map** listing which sleeping clients have buffered data; a client sleeps most of the time, wakes for the beacon, and if flagged sends a **PS-Poll** to fetch its frames. *Use case:* a battery sensor or phone that mostly idles and only occasionally receives data. Trade-off: more frequent beacons ⇒ less clock drift but less sleeping; rarer beacons ⇒ more sleep but more drift (and you may need a more reliable clock).
2. **APSD (Automatic Power-Save Delivery) / app-driven listening:** the device stays in low-power listening and only wakes to **send** at fixed intervals, prompting the AP to deliver buffered downlink then. *Use case:* periodic-reporting devices (VoIP, telemetry) where the **application** dictates when traffic flows, so power saving is effectively scheduled by the app.

D-MAC-AODV — Two MAC protocols that work well with AODV, and two that work poorly

Layers: Data Link & Network

Tags: `MAC-protocol` `AODV` `CSMA/CA` `BMAC` `TSMP` `TDMA` **Book:** CB · **Seen:** bank Q68; 2020, 2022

Hughes cross-layer favourite. AODV is **reactive/on-demand** with a **dynamic** topology, so it pairs well with **contention-based, schedule-free** MACs: - **Good — CSMA/CA and BMAC:** they transmit whenever needed without any global schedule or tight time sync, matching AODV's "set up a route only when I have a packet" nature and tolerating nodes joining/leaving. - **Poor — TSMP and (generic) TDMA:** these need a **pre-established, network-wide schedule and time synchronisation**. That clashes with AODV's on-demand, frequently-changing routes — every topology change would force costly re-scheduling, and a node can't get a slot to do route discovery before it's scheduled. Scheduled MACs assume a **stable** topology; AODV assumes the opposite.

D-FRAMELOSS — Sources of wireless frame loss; interaction with TCP congestion control; is packet loss a good congestion signal in 802.11?

Layers: Data Link, Network & Transport

Tags: `frame-loss` `802.11` `congestion-control` `TCP` `channel-hopping` `TSMP` **Book:** CB/OB · **Seen:** bank Q69 + Q103; 2022

Key sources of wireless frame loss at the data-link layer: interference from other transmitters and from badly-shielded devices (microwaves, motors) or signal-absorbing objects (water); **signal attenuation** (the signal weakens with distance until the receiver can't decode it); and **collisions** (hidden terminals). Protocols mitigate via gentle random back-off (ALOHA/CSMA-CA) and **channel hopping** (TSMP) against narrow-band interference.

Interaction with TCP congestion control + "is packet loss a good congestion signal here?" No — and this is the trap. TCP interprets packet loss as **congestion** and backs off.

But in 802.11 most loss is **interference / weak signal / collision**, **not** a full buffer. So TCP **wrongly throttles** on a perfectly uncongested (just noisy) link, tanking throughput. Worse, retransmissions over a lossy wireless hop may themselves be lost, adding overhead. This is exactly why **data-link-layer retransmission/ACKs on 802.11** (see T-RETRANS) are valuable — they hide link loss so TCP doesn't misread it as congestion.

D-ETHERNET — Hub vs switch; how a switch improves efficiency; when a hub is preferable

Layers: Data Link

Tags: `hub` `switch` `Ethernet` `CSMA/CD` `full-duplex` `promiscuous-mode` **Book:** CB · **Seen:** bank Q70 + Q73; 2014

A **hub** is a "dumb wire" that **broadcasts every incoming frame to all ports** → all hosts share one collision domain, bandwidth is wasted on unwanted frames, and **CSMA/CD** is needed (half-duplex). A **switch** is a fast PCB that **learns MAC addresses and forwards each frame only to the correct port**, giving each link its **own collision domain** and **full-duplex** operation (no CSMA/CD) → far better bandwidth use and improved privacy.

	Hub	Switch
Forwarding	broadcast to all ports	only to the destination port
Collision domain	one (shared)	one per port
Duplex / CSMA/CD	half-duplex, needs CSMA/CD	full-duplex, none
Efficiency / security	low / poor	high / better

When is a hub preferable (equal cost)? When you *want* every node to see all traffic — e.g. **network sniffing/monitoring, protocol analysis, or teaching/experiments** (running promiscuous-mode capture, observing ALOHA/CSMA behaviour). A switch's per-port isolation makes that impossible without port mirroring.

D-CABLELEN — Cable-length limits: switched vs classical Ethernet; why classical is slower with a long cable

Layers: Data Link & Physical

Tags: `cable-length` `CSMA/CD` `collision-detection` `Ethernet` `full-duplex` `half-duplex` **Book:** CB · **Seen:** bank Q71 + Q72; 2020

Classical Ethernet is **half-duplex** and uses **CSMA/CD**: to detect a collision, a frame's transmission time must be **≥ 2× the propagation delay** (a sender must still be transmitting when a far-end collision echoes back). A **longer cable** ⇒ longer propagation ⇒ you must make frames **longer** (carrier extension / minimum frame size) to stay detectable ⇒ **lower effective throughput**. So classical Ethernet's length is limited by **both collision detection and signal strength**.

Switched, full-duplex Ethernet has **no CSMA/CD** (each link is point-to-point, send and receive on separate pairs), so there are **no collisions** to detect and length is limited **only by signal attenuation**. Hence switched Ethernet doesn't share classical Ethernet's collision-driven cable-length limit. (Caveat: the *switch and cabling* must actually support full-duplex.)

D-PRIVSEC — Privacy & security: classical Ethernet vs switched Ethernet vs 802.11

Layers: Data Link

Tags: `promiscuous-mode` `hub` `switch` `Ethernet` `802.11` `WiFi` **Book:** CB · **Seen:** bank Q74; 2022-06-28

- **Classical (hub) Ethernet:** everyone on the shared medium sees **all** traffic → trivial eavesdropping via **promiscuous mode**. Serious if there's no end-to-end encryption.
- **Switched Ethernet:** the switch forwards frames only to the destination port, so a node **can't** see others' traffic by default → much safer. **But** if a **hub/sub-network hangs off a switch port**, everyone on that hub can still sniff each other (the classical problem returns locally), and tricks like MAC flooding/spoofing can still defeat it.
- **802.11 (WiFi):** anyone **in radio range** can receive the frames → inherently sniffable (sometimes useful, e.g. the NAV vector for coordination), so it **relies on encryption** (WPA2/3) for confidentiality.

Common thread: without **end-to-end (or link) encryption**, shared/broadcast media leak data.

D-BMAC — When does BMAC outperform TSMP? Configure BMAC for high vs low message rate

Layers: Data Link

Tags: `BMAC` `TSMP` `LPL` `preamble` `sampling-interval` `duty-cycle` **Book:** CB/OB · **Seen:** bank Q75/Q76 = Q104; 2015, 2021, 2022, **most-repeated DL question**

BMAC uses **Low Power Listening (LPL)**: a receiver mostly sleeps, briefly **samples** the channel periodically, and a sender prepends a **preamble** long enough that the receiver wakes during it and stays to receive. **TSMP** is scheduled (TDMA + time sync + channel hopping) and is excellent at steady sending/receiving but pays a **high join cost** — new nodes must listen and learn the schedule.

When BMAC outperforms TSMP: highly dynamic / sparse / long-distance networks where nodes **come and go** frequently or traffic is **sporadic** — BMAC just wakes and sends with no schedule to maintain, whereas TSMP's overhead of (re)learning slots dominates. (TSMP wins the opposite case: a **static, dense mesh** with steady periodic traffic and nodes close together.)

Configuration trade-off (preamble length ↔ sampling/wake interval): - **High message rate: short preamble + frequent sampling (short sample interval).** Sending is cheap (low per-message overhead) which matters when sending often; the receiver wakes often, but that cost is justified by the traffic. - **Low message rate: long preamble + infrequent sampling (long sample interval).** The receiver sleeps longer (cheap listening, saves battery on a mostly-idle node); the rare messages just carry a longer, more expensive preamble.

D-LPL — How does preamble length constrain the polling/sampling interval in BMAC?

Layers: Data Link

Tags: `LPL` `preamble` `sampling-interval` `BMAC` `duty-cycle` **Book:** CB · **Seen:** bank Q77; 2022-06-28

For LPL to work, the receiver must wake **at least once during every preamble**, so the **preamble length must be ≥ the sampling (polling) interval**. This is the core trade-off: a **longer preamble permits a longer sleep interval** (cheaper *listening*, but more expensive *sending*); a **shorter preamble forces more frequent polling** (cheaper *sending*, but more expensive *listening*). You tune the pair to the expected traffic rate (see D-BMAC).

D-TSMP — Describe TSMP; compare Slotted ALOHA and TSMP on time synchronisation

Layers: Data Link

Tags: `TSMP` `TDMA` `time-synchronization` `channel-hopping` `slotted-ALOHA` **Book:** CB · **Seen:** bank Q78; 2014, 2015

TSMP (Time Synchronized Mesh Protocol): a wireless **mesh** protocol for reliable, efficient low-power communication. All nodes share a **common clock** (from a **clock-master**) and use **TDMA** — each node has assigned **slots** for transmit/receive, so transmissions are **scheduled to avoid collisions**. It adds **channel hopping** to resist interference. Result: deterministic, low-collision, low-power — at the cost of needing synchronisation and a join/scheduling phase.

Slotted ALOHA vs TSMP (time synchronisation): | | Slotted ALOHA | TSMP | |—|—|—| | Time sync | yes — shared slot boundaries | yes — global clock via clock-master | | Carrier sense before sending | **no** | no (but slots are *assigned*, not contended) | | Collisions | **possible** (two nodes pick the same slot) | **avoided** (each node owns its slot) | | Pros | simple | reliable, deterministic, low power | | Cons | collisions under load | join/scheduling overhead, needs tight sync |

Both rely on synchronisation, but Slotted ALOHA only *aligns* contention (collisions still happen), whereas TSMP *schedules* it away.

D-COMBINE — Combine BMAC's LPL with TSMP for seamless coexistence

Layers: Data Link

Tags: `BMAC` `LPL` `TSMP` `TDMA` `time-synchronization` `duty-cycle` **Book:** OB · **Seen:** bank Q79

Use **TSMP's synchronised TDMA schedule as the backbone**, but apply **LPL within or around the slots** so radios still sleep aggressively: nodes wake just before their assigned slot (no long preamble needed for scheduled traffic), while a **BMAC-style long-preamble/LPL fallback** handles **unscheduled or newly-joining** nodes that aren't yet in the schedule. This gives TSMP's collision-free determinism for steady traffic **and** BMAC's low-power, schedule-free flexibility for sporadic/dynamic nodes — the synchronised slots prevent the two regimes from colliding.

D-STUFFING — Why byte stuffing works; a numerical example where bit stuffing is more efficient

Layers: Data Link

Tags: `byte-stuffing` `bit-stuffing` `framing` **Book:** CB · **Seen:** bank Q80; 2018, 2023

Framing marks frame boundaries with a special **flag**. The data might *contain* that flag pattern, so we escape it: - **Byte stuffing:** the boundary is a flag **byte**; whenever the flag byte (or the ESC byte itself) appears in the data, insert an **ESC** byte before it. The receiver removes ESCs to recover the data. It works because every accidental flag in the data is unambiguously marked, so only a *real* flag stands alone — but escaping whole bytes is **wasteful**. - **Bit stuffing:** the flag is the bit pattern **`01111110`**; while transmitting data, **after any run of five consecutive 1s, insert a 0** (unless it's a genuine frame flag). The receiver deletes the 0 after five 1s. It's **transparent** to upper layers and far cheaper.

Numerical example (bit stuffing more efficient): sending the byte `0xFF = 1111 1111` (eight 1s). - *Bit stuffing:* after the first five 1s insert one 0 → `11111 0 111` = **9 bits** (1 stuffed bit). - *Byte stuffing* (if `0xFF` were the flag): you'd insert a full **8-bit** ESC byte → **16 bits** for the same payload byte. So bit stuffing adds **1 bit** where byte stuffing adds **8** — much less overhead.

D-SW — Stop-and-wait: which medium is poor / good, and estimate throughput

Layers: Data Link, Physical & Transport

Tags: `stop-and-wait` `RTT` `throughput` `bandwidth-delay-product` `satellite` `Ethernet` `sliding-window` **Book:** CB/OB · **Seen:** bank Q38 + Q81/Q82 = Q105; 2018, 2021, 2023

Stop-and-wait sends **one frame, then waits a full RTT** for its ACK before sending the next. So throughput ≈ **one frame per RTT**, regardless of how fast the link is. - **Performs poorly: high-RTT** media — classically a **geostationary satellite** link (RTT ~500 ms+). The link may be fast, but you sit idle most of each RTT. Worst case is a **high-bandwidth, high-latency** link (large **bandwidth-delay product**): you could have many frames in flight

but send only one. - **Performs well: low-RTT** media — e.g. a short **Gigabit Ethernet** link (RTT ~tiny) or a short copper run, where one-frame-per-RTT still keeps the link nearly full and the bookkeeping cost is negligible relative to send time.

Throughput estimate ≈ `frame_size / RTT` (when RTT dominates): - GEO satellite: 1000-byte frame, RTT 500 ms → 8000 bits / 0.5 s = **16 kbps**. - Fast LAN: 1500-byte frame, RTT 10 ms → 12000 bits / 0.01 s = **1.2 Mbps**.

Best achievable: 1 frame / RTT — to do better you need a **sliding window** (many frames in flight per RTT). **Hughes framing:** stop-and-wait is fine only when RTT is low and the per-frame send cost is large relative to the bookkeeping.

D-LORA — LoRa: problems, best/worst use, MAC utilisation, and an improved MAC

Layers: Data Link & Physical

Tags: `LoRa` `ALOHA` `duty-cycle` `TDMA` `CSMA/CA` `channel-hopping` `time-synchronization` `IoT`
Book: CB/OB · **Seen:** bank Q83/Q84 = Q106; 2024-closed, 2025-06-10

LoRa: long-range (~20 km), low-power IoT; packets can take **>1 s** to transmit; **no network time / synchronisation**; **anyone can deploy a gateway unlicensed**.

Problems / when it performs poorly: the MAC is essentially **ALOHA-like** (transmit when ready, no sensing, no sync), so under **dense / high-traffic** conditions collisions are frequent and, because **airtime is over a second**, **each collision is very expensive** (huge wasted time) and latency is high; the **unlicensed, uncoordinated gateways** share spectrum with no scheduling → interference and no scalability or security guarantees. **Low-latency** applications are a bad fit.

When it performs best: sparse, low-duty-cycle, delay-tolerant, long-range sensing — a few devices sending small infrequent readings (agriculture, metering, environmental sensors) where the long range and low power matter and occasional collisions/latency don't.

Expected channel utilisation: low — like **Pure ALOHA, only ~18%** (no carrier sense, no synchronisation), and worse in practice given the long airtime.

Improved MAC (use Data-Link techniques): add **time synchronisation + slotting/TDMA** (TSMP-style) so devices transmit in assigned slots instead of blindly; or add **CSMA / Listen-Before-Talk** to cut blind collisions; add **channel hopping** and **adaptive data-rate** to fight interference and shorten airtime where possible; add **duty-cycle coordination** across gateways; and add **acknowledgements/retransmission** for reliability. Each directly attacks a listed weakness (collisions, long airtime cost, gateway uncoordination).

D-NULLMAC — A "Null MAC" (listen 100%, send immediately, no retransmission): problems

Layers: Data Link

Tags: `null-MAC` `MAC-protocol` `ALOHA` `collision-detection` **Book:** CB · **Seen:** 2015-23 (older; not in 106 bank)

With **no carrier sense / no back-off and no retransmission**: under **high traffic** many frames collide, and because nothing is ever re-sent, **every collided frame is lost forever** — there's no recovery, so as load rises, **goodput collapses**. In a **crowded city centre** (many dense, competing transmitters) the same happens, amplified: constant collisions, no recovery, near-zero useful throughput. The only thing saving it at *light* load is that messages are spread out in time — fine when traffic is sparse, useless when it isn't.

D-PADDING — Ethernet frame padding: why, and two cases

Layers: Data Link & Physical

Tags: `padding` `Ethernet` `CSMA/CD` `collision-detection` `cable-length` **Book:** CB · **Seen:** 2015-06-15 (older; not in 106 bank)

Classical Ethernet enforces a **minimum frame size (64 bytes)**: a frame must be long enough that the sender is **still transmitting** when a collision from the far end of the maximum-length cable returns, otherwise **CSMA/CD can't detect the collision**. So short frames are **padded** up to the minimum. Two cases where padding kicks in: (1) a payload **smaller than the minimum** (e.g. a tiny ACK/control frame) gets padded to 64 bytes; (2) on a **longer cable / higher speed** (Gigabit), the minimum effective frame is extended further (**carrier extension**) so collision detection still works over the larger propagation window — directly linking to why classical Ethernet slows down on long cables (see D-CABLELEN).